

System Design: Metering & Billing Capstone

Problem Statement: We are building a robust SaaS billing backend using Node.js, Express, and PostgreSQL. The service needs to record billable usage events for tenants, strictly enforce plan quotas, securely calculate monthly costs from usage using integer-based math, and synchronize subscription statuses via Stripe test-mode webhooks. The system must be highly reliable, ensuring correct behavior under edge cases like network retries, quota boundaries, and duplicate webhooks.

Note: This system will not handle live payments or process real credit card data. All billing and payment interactions will strictly use Stripe's test mode. Complex features like prorated plan downgrades, custom negotiated pricing tiers, and multi-currency support are also out of scope to keep the system simple and focused on core metering correctness.

1. Database Schema

The core database schema consists of four main tables. Here is the structure and sample data for each:

Table Schemas:

tenants		
Column Name	Column Type	Sample Value
id	(UUID, Primary Key)	550e8400-e29b
name	(VARCHAR)	Acme Corp
created_at	(TIMESTAMP)	2026-08-13T10:00:00Z

plans		
Column Name	Column Type	Sample Value
id	(VARCHAR, Primary Key)	free
name	(VARCHAR)	Free Plan
monthly_token_quota	(INTEGER)	10000

<i>subscriptions</i>		
Column Name	Column Type	Sample Value
id	(UUID, Primary Key)	123e4567-e89b
tenant_id	(UUID, Foreign Key to tenants.id)	550e8400-e29b
plan_id	(VARCHAR, Foreign Key to plans.id)	pro
status	(VARCHAR)	active
stripe_customer_id	(VARCHAR, Nullable)	cus_test123
stripe_subscription_id	(VARCHAR, Nullable)	sub_test456
created_at	(TIMESTAMP)	2026-08-13T10:05:00Z

<i>usage_events</i>		
Column Name	Column Type	Sample Value
id	(UUID, Primary Key)	123e4567-e89b
tenant_id	(UUID, Foreign Key to tenants.id)	550e8400-e29b
idempotency_key	(VARCHAR, Required)	req-idx-001-abc
input_tokens	(INTEGER)	150
output_tokens	(INTEGER)	50
created_at	(TIMESTAMP)	2026-08-13T10:10:00Z

Constraint: UNIQUE (tenant_id, idempotency_key) to prevent duplicate events.

2. Plans + Quotas

The system defines two default plans to demonstrate quota boundaries:

- *Free Plan* (*plan_id*: 'free')
 - **Quota**: 10,000 total tokens per billing cycle
 - **Behaviour**: Once 10,000 tokens are consumed, further requests are rejected with a **429 Too Many Requests** status code. The tenant must upgrade to continue.
- *Pro Plan* (*plan_id*: 'pro')
 - **Quota**: 1,000,000 total tokens per billing cycle.
 - **Behavior**: High limit plan accessible via a Stripe Checkout upgrade. Standard **429** logic applies if this limit is reached.

3. Metering API Contract

The system will expose a simple REST API for billable actions and reporting:

→ **POST /api/v1/generate** (Billable Endpoint)

- **Purpose**: Simulates a billable action (e.g., an LLM generation request).
- **Headers**:
 - **Idempotency-Key** (String, Required) - UUID generated by the client.
 - **X-Tenant-Id** (String, Required) - ID of the tenant making the request.
- **Request Body** (JSON):

```
{
  "input_tokens": 150,
  "output_tokens": 350
}
```

- **Success Response (200 OK):**

```
{
  "success": true,
  "event_id": "uuid-here",
  "tokens_used": 500
}
```

- **Error Responses:**
 - 429 Too Many Requests:
 - {"error": "Quota exceeded. Upgrade your plan."}
 - 402 Payment Required:
 - {"error": "Subscription inactive."}

→ **POST /api/v1/generate** (Billable Endpoint)

- **Purpose:** Retrieves the tenant's current usage, plan limits, and calculated cost.
- **Headers:** X-Tenant-Id (String, Required)
- **Success Response (200 OK):**

```
{
  "plan": "free",
  "total_tokens_used": 8500,
  "quota_limit": 10000,
  "current_cost_cents": 0
}
```

4. Architecture Layers

The Express application will follow a clean, layered architecture:

1. **Route Handlers (Controllers):** Handle HTTP requests, extract parameters/headers, and return HTTP responses with correct status codes.
2. **Service Layer:** Contains core business logic (cost calculation, quota enforcement, idempotency checks). This layer is completely agnostic of HTTP and Express details.
3. **Repository/Database Layer:** Manages all interactions with the PostgreSQL database. Responsible for executing queries and ensuring data integrity.

5. Idempotency Strategy

To safely handle network retries (e.g., if a client loses connection before receiving a response), the POST /generate endpoint relies on strict idempotency enforcement at the database level.

- **Database Constraint:** The usage_events table has a UNIQUE(tenant_id, idempotency_key) constraint.
- **Request Flow:**
 1. When a request arrives, the service calculates if it's within quota, then attempts to insert the new usage_event into PostgreSQL.
 2. If the Idempotency-Key was already used by this tenant, PostgreSQL throws a unique constraint violation error.
 3. The service catches this specific database error, skips creating a new event, and queries the existing event to return a successful 200 OK response.
- **Outcome:** This guarantees that a request + idempotency key results in exactly one billable usage event, eliminating the risk of double-charging due to network retries.

5. Quota Enforcement & Boundary Rules

Quota checks happen securely in the service layer before any billable action is executed.

- **Within Limit:** Request succeeds, usage event is recorded.
- **At the Boundary:** If a request attempts to consume more tokens than the remaining quota allows, the entire request is rejected (429). We do not partially fulfill requests.
- **Over Limit:** Rejected immediately with 429.

6. Architecture Trade-offs

- ❖ **Monolithic Express App vs. Microservices:** We are choosing a layered monolith. While microservices offer independent scaling, they introduce immense operational complexity. A well-structured monolith is significantly simpler to build, test, and reason about.
- ❖ **PostgreSQL for Everything vs. Redis + Postgres:** We will use PostgreSQL to store usage events and enforce idempotency using unique constraints. While Redis is faster for rate-limiting and caching idempotency keys, introducing it adds another infrastructure dependency. Relying solely on Postgres leverages its strong ACID guarantees for correctness without extra complexity.